

CSA0624 - Design and Analysis of Algorithms

CO2_AT2 Scenario-Based Assessment

Name: Aryan Arun

Reg.No: 192411157

Q1. Sorting Nearly-Sorted Student Records

Scenario: Student marks are almost sorted except for a few recently updated entries, and the system must re-sort frequently with minimal computation time. The choice is between Selection Sort, Bubble Sort, and Insertion Sort.

a) Analysis of All Three Algorithms in This Context

Selection Sort: On every pass, it scans the entire unsorted segment to find the minimum, regardless of how the data is arranged. It is non-adaptive — it performs exactly $n(n-1)/2$ comparisons whether the array is already sorted or completely reversed. Its only advantage is a low, bounded number of swaps (at most $n-1$). For nearly-sorted data, this is wasteful: the algorithm does not recognize that most elements are already in place and still re-scans everything.

Bubble Sort: In its optimized form (with a flag that stops early when no swaps occur in a pass), Bubble Sort is adaptive to some extent — if the array is already sorted, it detects this in a single pass, giving $O(n)$. However, Bubble Sort only swaps adjacent elements, so a misplaced record must 'bubble' one position at a time toward its correct location. If an updated entry is far from its correct position, this still requires many passes, making its performance sensitive to how far out of place elements are, but generally less efficient than Insertion Sort for the same displacement.

Insertion Sort: This is the most adaptive of the three. It builds the sorted list one element at a time, shifting only the elements that are genuinely out of place. Its running time is proportional to the number of inversions (pairs of out-of-order elements) rather than n^2 — for data that is already sorted except for a few entries, the number of inversions is small, so Insertion Sort runs in close to $O(n)$ time. A new/updated entry can be 'inserted' directly into its correct position in one pass, without disturbing already-sorted elements.

b) Recommendation and Justification

Algorithm	Comparisons (nearly sorted)	Swaps/Shifts (nearly sorted)	Adaptive?
Selection Sort	Always $n(n-1)/2$ — unaffected by order	$\leq n-1$, but always performed regardless of need	No
Bubble Sort (optimized)	$O(n)$ if sorted, but grows if elements are far displaced (moves 1 position per pass)	Proportional to displacement distance, one step at a time	Partially
Insertion Sort	$O(n + d)$, where d = number of inversions — very low here	Only shifts elements that are actually out of place	Yes — fully adaptive

Recommended algorithm: Insertion Sort. Because only a few records are updated, the number of inversions in the marks list stays small regardless of the total record count n . Insertion Sort exploits this directly: its comparison and shift counts scale with the number of out-of-place elements, not with n^2 , so it approaches $O(n)$ performance in this scenario. It also allows incremental re-sorting — when a record is updated, only that entry needs to be re-inserted into its correct position rather than re-sorting the entire list — which matches the requirement for minimal, frequent computation. Selection Sort is ruled out because it ignores existing order entirely, and Bubble Sort, while better than Selection Sort, still corrects positions only one adjacent step at a time, making it less efficient than Insertion Sort's direct placement when the same small set of elements are moved.

Q2. Closest Pair Among 500 Warehouse Locations

Scenario: A logistics company must find the closest pair among 500 warehouse locations to optimize delivery routes, using a brute-force distance comparison. The system slows down as more warehouses are added.

a) How the Brute-Force Approach Works

The brute-force closest-pair method computes the Euclidean distance between every possible pair of warehouses and keeps track of the minimum distance found so far. For each pair of warehouses (W_i, W_j), the distance is calculated as $d = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$. Every one of the $C(500, 2)$ possible pairs is generated and evaluated exactly once, and the pair yielding the smallest distance is reported as the answer. No assumptions are made about the spatial layout — every pair is checked exhaustively, which guarantees correctness but does no work to avoid unnecessary comparisons.

b) Deriving the Time Complexity

For n warehouse locations, the number of distinct pairs is $C(n, 2) = n(n-1)/2$. For each pair, computing the distance takes constant time $O(1)$ (a fixed number of arithmetic operations). Therefore the total time is:

$$T(n) = n(n-1)/2 \times O(1) = O(n^2)$$

Substituting $n = 500$: number of pairs = $500 \times 499 / 2 = 124,750$ distance calculations, plus roughly 124,749 comparisons to track the running minimum — about 250,000 basic operations in total. Space complexity is $O(1)$ beyond storing the input coordinates, since only the current minimum distance and pair need to be retained.

c) Why Scalability is an Issue

Because the algorithm is $O(n^2)$, the work needed grows with the square of the number of warehouses, not linearly:

Number of Warehouses (n)	Approximate Pairs Evaluated ($n^2/2$)
500	$\approx 125,000$
1,000 (2 $\times$)	$\approx 500,000$ ($\approx 4\times$)
5,000 (10 $\times$)	$\approx 12,500,000$ ($\approx 100\times$)
10,000 (20 $\times$)	$\approx 50,000,000$ ($\approx 400\times$)

Doubling the number of warehouses roughly quadruples the computation time, and a tenfold increase in warehouses results in roughly a hundredfold increase in work. This quadratic blow-up means the brute-force approach quickly becomes impractical as the logistics network expands — a computation that takes a fraction of a second at $n = 500$ could take minutes or hours at $n = 50,000$, making it unsuitable for frequent, real-time route optimization. For large-scale deployments, a divide-and-conquer closest-pair algorithm ($O(n \log n)$) or spatial indexing structures such as k-d trees, Voronoi diagrams, or grid-based spatial partitioning would be used instead, since they avoid comparing every pair and instead limit comparisons to nearby candidates.

Q3. Sequential Search on an Unsorted Hospital Patient Database

Scenario: Patient records are stored unsorted because records are frequently added and updated, and doctors need to quickly search for patient details during emergencies.

a) How Sequential (Linear) Search Works Here

When a doctor searches for a patient (for example, by patient ID or name), the system starts at the first record and compares the search key against each record in turn, moving sequentially through the list until either a matching record is found or the entire list has been scanned without a match. Because the list is unsorted, this is the only search method directly usable without additional preprocessing — it makes no assumption about the order of records, so it works correctly no matter how recently a record was added or updated.

b) Best, Worst, and Average Case Performance

Case	Scenario	Complexity
Best Case	The requested patient's record happens to be the very first one checked.	$O(1)$
Worst Case	The record is the last one in the list, or the patient is not found at all (the entire list must be scanned).	$O(n)$
Average Case	On average, about half of the n records are checked before a match is found.	$O(n/2) \rightarrow O(n)$

In an emergency setting, the worst case is the operationally important one: if a record is near the end of the list (e.g., a very recently added patient), the search could take proportionally longer as the database grows, which is a genuine risk in time-critical situations.

c) Why Sorting is Not Preferred Here

- High update frequency: hospital records are constantly inserted and modified (new admissions, updated vitals, discharge status). Keeping the list sorted would require re-sorting or shifting elements on every insertion/update, costing $O(n)$ per update for an array-based structure — this overhead can exceed the benefit of faster searches, especially if searches are not performed as often as updates.
- Correctness under urgency: in an emergency, staff cannot wait for a background sort operation to complete or risk searching a database that is only partially re-sorted after a rapid update.
- Low benefit-to-cost ratio for this workload: Binary Search's $O(\log n)$ advantage over Linear Search's $O(n)$ only pays off if the sorted order can be maintained cheaply and searches happen far more often than updates — in this hospital scenario, updates are frequent enough that the sorting overhead is not justified.
- Better alternatives exist: rather than sorting purely to speed up search, hospitals typically use hash-based indexing (e.g., indexing by patient ID) or database indexes (B-trees) that support both fast lookup and fast updates — but within the simple sorted-vs-unsorted comparison implied by this scenario, unsorted storage with sequential search remains the more practical choice given the update-heavy, order-agnostic nature of the data.

Q4. Brute-Force Convex Hull in a GIS Application

Scenario: A GIS (Geographic Information System) application determines the boundary of a region by identifying the outermost points using a brute-force convex hull approach.

a) How the Brute-Force Convex Hull Algorithm Works

The algorithm treats every pair of points as a candidate edge of the boundary. For each pair of points (P_i, P_j), it draws the line through them and checks the position (left/right, via the sign of the cross product) of every other point relative to that line. If all remaining points lie on the same side of the line (or exactly on it), the segment P_iP_j is confirmed as an edge of the convex hull — meaning no point of the region lies outside that boundary line. If points are found on both sides, the segment is discarded as an interior chord rather than a boundary edge. Once every pair has been tested, the accepted edges are connected in order to trace the complete convex boundary of the region.

b) Computational Complexity Analysis

For n points, the number of candidate point pairs is $C(n,2) = n(n-1)/2$, which is $O(n^2)$. For each candidate pair, the algorithm must check the remaining $(n-2)$ points to see which side of the line they fall on, which is $O(n)$ work per pair. Combining these:

Total time = $O(n^2)$ pairs $\times$ $O(n)$ checks per pair = $O(n^3)$

This cubic growth means the brute-force convex hull algorithm is computationally expensive even for moderately sized datasets, though it remains simple, exhaustive, and reliably correct — useful for small test regions or for validating faster algorithms.

c) Why It Becomes Inefficient for Large Datasets

- GIS datasets often contain thousands to millions of coordinate points (e.g., satellite terrain data, survey points, or map boundary data), and $O(n^3)$ growth quickly becomes computationally infeasible — for example, $n = 10,000$ points would require on the order of 10^{12} operations, far beyond practical real-time processing.
- Every additional point disproportionately increases the workload: doubling the number of points increases the work roughly eightfold (2^3), unlike linear or log-linear algorithms whose workload grows much more gently.
- Much of the $O(n^3)$ work is redundant — most candidate edges are quickly seen to be interior chords, but the brute-force method still fully tests every point against every candidate line rather than eliminating clearly interior points early.
- More scalable algorithms exist and are preferred for large datasets: Graham's Scan and Divide-and-Conquer Convex Hull both achieve $O(n \log n)$, and Quickhull achieves $O(n \log n)$ on average — all of which discard interior points far more efficiently, making them the practical choice for real-world GIS-scale data.

Q5. Brute-Force String Matching for Plagiarism Detection

Scenario: A plagiarism detection system compares a pattern (a suspected copied phrase) against a very large document to find matches, using brute-force string matching, where the pattern appears rarely.

a) How Brute-Force String Matching Works

Let the document (text) have length n and the pattern have length m . The algorithm aligns the pattern with the text starting at position 0, then compares the pattern's characters one by one against the corresponding text characters. If all m characters match, an occurrence of the pattern has been found at that position. If a mismatch occurs at any character, the comparison for that alignment stops immediately, and the pattern is shifted one position to the right in the text, after which the comparison process restarts from the first character of the pattern. This continues until the pattern has been tested at every possible starting position in the text, from position 0 to position $n-m$.

b) Time Complexity Analysis

Case	Description	Complexity
Best Case	Mismatches occur at the very first character of the pattern for almost every alignment, so each attempt is discarded quickly.	$O(n)$
Worst Case	For every one of the $(n-m+1)$ alignments, up to m characters must be compared before a mismatch (or match) is confirmed — e.g., highly repetitive text similar to the pattern.	$O((n-m+1) \times m) \approx O(n \cdot m)$
Average Case	Depends on the alphabet size and text structure; generally closer to $O(n)$ for typical natural-language text with a reasonably large alphabet.	$O(n)$ typical, up to $O(n \cdot m)$ worst case

c) Why Performance Degrades in This Scenario

- The document length n is very large, so even the linear factor n in the complexity expressions represents a large absolute amount of work; since brute-force matching re-examines the text from scratch at each shifted position, this cost accumulates across the entire document.
- Because the pattern shifts by only one position at a time (no skipping), the algorithm cannot take advantage of information learned from a failed comparison — even if seven characters matched before a mismatch, that information is discarded, and the next alignment starts comparing from scratch, unlike smarter algorithms that use this information to skip ahead.
- Because the pattern appears rarely, the vast majority of the $(n-m+1)$ alignments end in a mismatch rather than a full match — meaning nearly the entire $O(n \cdot m)$ worst-case workload may be spent on failed attempts rather than productive matching, and if the document contains sections with partial similarity to the pattern (e.g., repeated words or common phrases), many alignments may require several character comparisons before failing, pushing performance toward the $O(n \cdot m)$ worst case rather than the more optimistic $O(n)$ case.
- For large-scale plagiarism detection across many long documents, this inefficiency compounds, since the same brute-force scan must be repeated for every pattern (suspected phrase) checked against every document.
- More efficient string-matching algorithms are preferred at scale: the Knuth-Morris-Pratt (KMP) algorithm and Boyer-Moore algorithm both achieve $O(n + m)$ by using information from previous comparisons to skip ahead intelligently, and the Rabin-Karp algorithm (using hashing) is particularly well suited to plagiarism detection since it can efficiently check many candidate patterns against the same document.

Summary Table

Q	Scenario	Best-Fit Technique / Key Complexity
1	Nearly-sorted student records	Insertion Sort — adaptive, $\approx O(n)$ for few inversions
2	Closest pair, 500 warehouses	Brute Force $O(n^2)$; scalability issue as n grows
3	Unsorted hospital records, emergency search	Sequential Search $O(n)$; sorting overhead not justified
4	GIS region boundary	Brute-Force Convex Hull $O(n^3)$; inefficient at scale
5	Plagiarism detection string matching	Brute-Force $O(n \cdot m)$ worst case; KMP/Boyer-Moore preferred at scale